

Assignment 1: Process, Thread, Scheduling

- Upload your PDF report named “HW1_Group[*your_group_number*].pdf”.
 - Specify each group member's name in the report.
 - For each programming exercise, submit a zip folder containing:
 - All source codes
 - The Makefile file that supports ‘make all’ ‘make clean’ and ‘make run’
 - By submitting a report, all group members acknowledge that they understand the answers to all questions and can demonstrate them when asked.
-

Q1: Process Creation

1. How many processes in total will be created by the following code?
2. How will you verify the number of processes created in this program? Include a screenshot of your verification in the report.
3. How many processes are created as a function of N?

```
4  #define N 3
5
6  int main () {
7
8      for (int i=0;i<N;i++)
9      {
10         fork();
11         fork();
12     }
13
14     return 0;
15 }
```

Q2: Process statuses

Analyze the following code. Compile and run it on your Linux system.

1. How many processes are created in total? Which process will sleep for 10s?
2. Are there any zombie and orphan process?
3. How will you check the status of the processes? What statuses do you see for the parent process? What statuses do you see for the child process? Include the screenshots of these process statuses.

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5
6      int pid = fork();
7      if(pid == 0)
8          printf("I'm the child %d\n", getpid());
9      else{
10         printf("My child is called %d\n", pid);
11         sleep(10);
12     }
13
14     printf("PID%d says goodbye\n", getpid());
15     return 0;
16 }
```

Q3: multi-threaded sum

Write a C program that uses multiple pthreads to perform the sum of an array. You can modify the source code skeleton “hw1-parallel-sum.c” to fulfill the following requirements:

- The master thread performs a serial sum first. It creates an array of 1,000,301 random floats in the [0,1] range and prints out the sum.
 - A group of worker threads are created. Each thread has a unique ID in [0, num_threads–1], where num_threads is given by argv[1].
 - Each thread sums up a portion of the array (initialized by the master thread) in parallel. The final sum is aggregated by the master thread and printed out.
 - Print two timings. One measures the serial sum and one for the parallel sum.
1. Run your program with 1, 2, 4 threads. Include the screenshots in the report.
 2. Which data structures may have read access from multiple threads?
 3. Which data structures may have read and write access from multiple threads?
 4. Name one or more scheduling algorithm suitable for this program. Name one or more scheduling algorithm NOT suitable for this program. Explain why.

Q4: multi-threaded histogram

Write a C program that uses multiple pthreads to build the histogram of an array. You can modify the source code used in Q3 to fulfill the following requirements:

- Create an array. The array length is given by argv[2]. The array should be initialized with random floats in the [0.0, 1.0] range.
 - The master thread builds the histogram of the array in argv[3] bins. Print the histogram.
 - A group of worker threads are created. The number of threads is given by argv[1]. Each thread builds the histogram of a portion of the array. The final histogram is built by aggregating these partial histograms. Print the histogram.
 - Print two timings. One measures the serial histogram and one for the parallel histogram.
1. Run: ./program 3 800 16, ./program 3 80000 16, and ./program 3 8000000 16. Include screenshots of your output here.
 2. Which data structures may have read access from multiple threads?
 3. Which data structures may have read and write access from multiple threads?
 4. Name one or more scheduling algorithm suitable for this program. Name one or more scheduling algorithm NOT suitable for this program. Explain why.
-

Q5: Scheduling Algorithms

Consider the following set of processes. A smaller priority number implies a higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	10	1
P2	1	3	3
P3	2	5	2
P4	6	1	0

For each of SJF, preemptive SRTF, and non-preemptive priority scheduling algorithms:

1. Draw the Gantt chart to illustrate the execution of these processes.
 2. Calculate the turnaround and waiting time for each process.
 3. Which algorithm results in the minimum average waiting time?
-

Q6: Optimizing Scheduling Algorithms

You will modify the source code skeleton “hw1-scheduling.c” to complete the 3 //TODOs in the code.

1. Is this a preemptive or non-preemptive scheduling? Explain why.
 2. Run the code and include screenshots.
 3. If the system needs to reduce thread 6’s turnaround time to be < 20. Is it feasible? If so, modify the code and show the updated screenshot. If not, explain why. *Hint: techniques such as aging and time quantum may be helpful.*
-

Q7: IPC, Context Switch, Kernel module

- You will modify the source code skeleton “hw1-ipc.c” to complete the //TODOs in the code. This C program creates pipes for inter-process communication between parent and child processes. The communication is repeated for ITTERS times.
- You will use “strace” to trace the syscalls behind the IPC program. Use strace to follow child processes, filter to the relevant system calls such as read() and write(), and print a summary of total counts per traced system call.

- You will compile and load a kernel module “hw1-csprobe.c” that probes context switches in a target process with PID. You will then use this kernel module to track context switch in the ipc program
1. In “hw1-ipc.c”, how do you close the ends of the two pipes that CHILD process never used?
 2. Write the full command you used strace for tracing. Include a screenshot of its printed summary of total counts per traced system call.
 3. From the strace output, which system call dominate the time. Explain possible causes for this.
 4. Write the full command you used for loading and unloading the csprobe kernel module. Include a screenshot of dmesg that capturing this kernel module’s activities.
 5. Write the steps you used to use csprobe kernel module to track the CHILD process of the hw1-ipc program. Include three screenshots that prints out “/proc/csprobe” at different time of one execution.
 6. Run the experiment in 5 a few times using different values for ITTERS. Do you observe any pattern of the voluntary and involuntary context switches?